

Hardware Implementation of Neural Network Inference Using SystemVerilog - Phase 2: Fixed-Point Neural Computation

1. Introduction

This phase continues our previous work with designing and implementing a neural network and converting those trained parameters to fixed-point for further inferences in SystemVerilog. To accomplish this, a shallow neural network was implemented from scratch in Python. The trained parameters were extracted and converted to fixed-point format for use in SV. The dataset used for this project is called the Wisconsin Breast Cancer Dataset (WBCD), which can be imported from the ``sklearn.datasets`` package.

This phase consists of three sections. Section one focuses on Data Selection, with a primary focus on analyzing the dataset, deriving useful information through statistical analysis and visualization, and selecting the best feature pair based on computed metrics and measurements.

Section two focuses on implementing the Neural Network Class from scratch and training it using the two selected, scaled features, and retrieving the trained parameters for further quantization investigations.

Finally, in section three, the importance of fixed-point representation in SystemVerilog will be discussed, and parameters will be converted using an appropriate quantization scale.

2. Data Analysis

This section starts by displaying a sample of data, providing a descriptive summary, and visualizing data. We will then proceed with selecting the best features and performing feature engineering.

2.1 Dataset

The first step involves loading in and analyzing the data using ``Pandas``, ``NumPy``, ``Matplotlib``, and ``Seaborn``. The original data has about 30 features and 569 records (examples).

A sample of the data is shown below:

mean radius	mean texture	worst smoothness	worst texture	worst perimeter
17.99	10.38	0.1622	17.33	184.60
20.57	17.77	0.1238	23.41	158.80
19.69	21.25	0.1444	25.53	152.50

Table 1. A sample of Data

2.2 Descriptive Summary

The number of null rows has been checked using `data.info()`. It is worth noting that no NaN values were found. Also, a descriptive summary showed the distribution of the features using `data.describe()` method.

2.3 Correlation Analysis

Since 30 features are too much for a small network with the architecture $2 \rightarrow 4 \rightarrow 2$, only the two most correlated features have been utilized as the input. The objective is to identify features with low pairwise correlation and high Pearson correlation with the target. A heatmap has been plotted to show the correlation between features and the target. (Fig. 1)

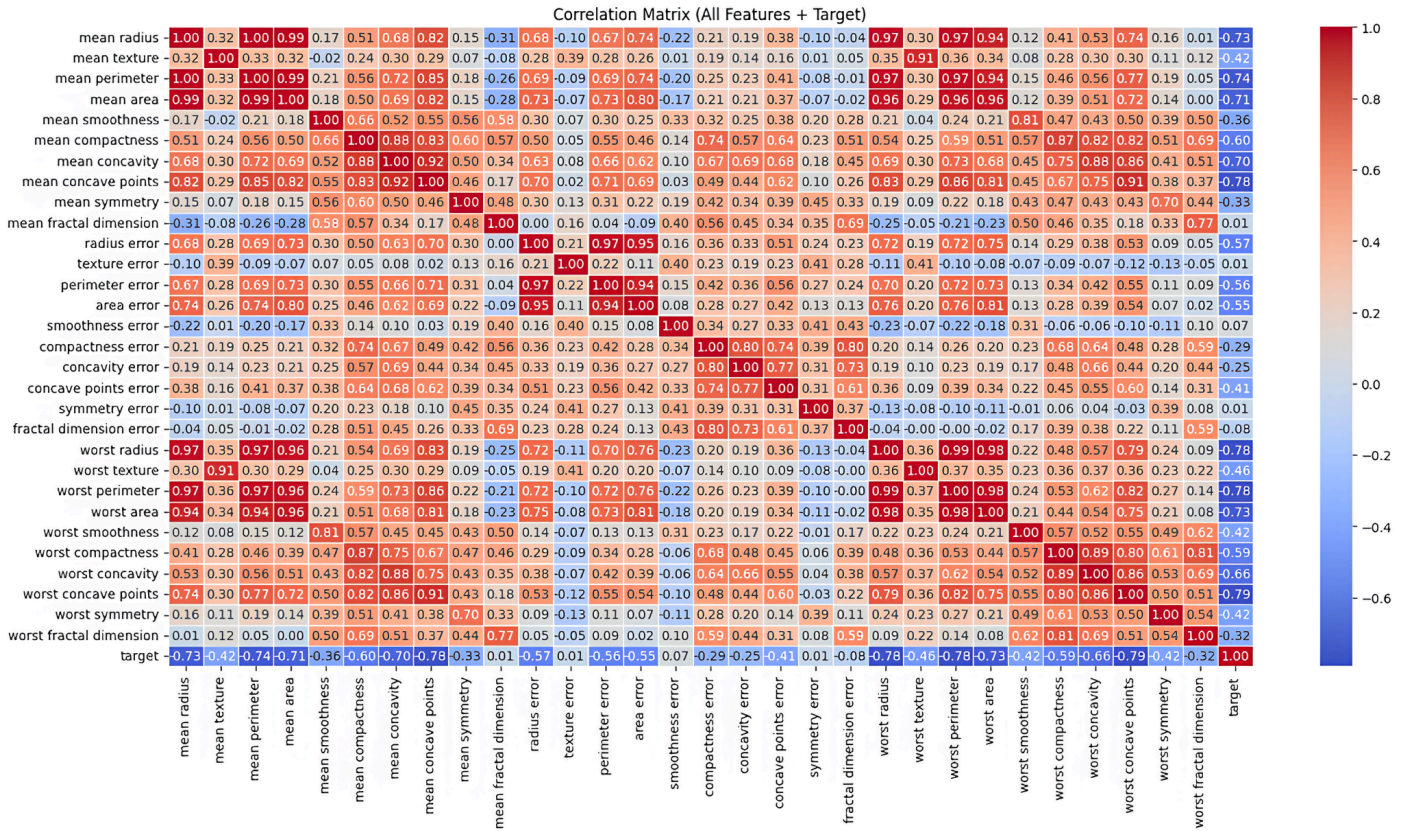


Figure 1. Correlation Heatmap

2.4 Feature Selection

Since choosing the two most correlated features based on only a Heatmap would be a difficult choice, quantitative metrics such as **Pearson correlation** (from the 'SciPy' package), and the **feature-pair scoring** method have also been utilized to make sure the selected features are scientifically reasonable.

The feature-pair scoring has been computed using the following formula:

$$score = |corr(A, y)| |corr(B, y)| (1 - |corr(A, B)|)$$

Where:

- $corr(A, y)$: correlation between feature A and target
- $corr(B, y)$: correlation between feature B and target
- $corr(A, B)$: correlation between the two features
- λ : penalty strength for redundancy

Top 3 features with the lowest multicollinearity and highest Pearson correlation:

feature_1	feature_2	target_corr_1	target_corr_2	feature_corr	score
mean radius	worst smoothness	0.730029	0.421465	0.119616	0.270878
mean perimeter	worst smoothness	0.742636	0.421465	0.150549	0.265874
mean area	worst smoothness	0.708984	0.421465	0.123523	0.261902

Table 2. Correlation Map between Features and Target

“*worst smoothness*” appears alongside other features in all three sets. The pair of features in the first row has the best performance, since the score is higher and the feature_corr is lower. Higher scores indicate feature pairs that simultaneously reveal strong correlations with the target and relatively low redundancy with each other. It is worth mentioning that for further analysis, training the network with all three pairs might end up in a more accurate result. This experiment is especially worth it in selecting the best pair of features. As an example, *mean perimeter* in the second pair has a higher correlation than the *mean radius* in the first pair. But this matter should also take into consideration that the higher the correlation of the features, the lower the final score. Therefore, selecting the best pair of features is actually a trade-off that should be handled in further analysis.

2.5 Outlier Analysis

Further visualization techniques, such as box plots, revealed that the selected features include outliers, which are data points that fall outside a defined range. Neural Networks can be sensitive to outliers, especially when training data are limited. Therefore, further analysis has been conducted, and various functions have been implemented to detect them. Since several outliers appear to belong to clinically relevant classes rather than measurement errors, they were retained to avoid discarding potentially informative observations.

A box plot is shown below (Fig. 2):

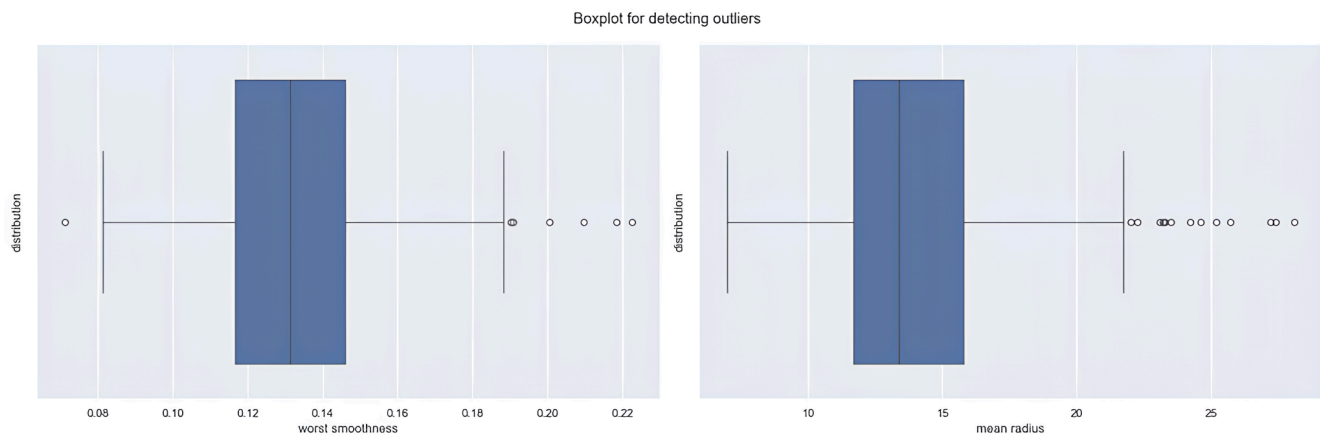


Figure 2. Boxplot for Displaying Data Distribution

2.6 Feature Scaling

To address the differences in variable ranges as shown in Table 1, a Scaling method, `MinMaxScaler()`, has been applied to the features after splitting them into train-test sets, ensuring they fall within the range of $[-1, 1]$.

Info	worst smoothness	mean radius
count	569	569
mean	0.132369	14.127292
std	0.022832	3.524049
min	0.071170	6.981000
25%	0.116600	11.700000
50%	0.131300	13.370000
75%	0.146000	15.780000
max	0.222600	28.110000

Table 3. Descriptive Info of Features

3. Model Implementation

This section continues with implementing a neural network from scratch using Python and NumPy. We will go through data splitting, scaling methods, tensor conversion, and network architecture.

3.1 Data Splitting

The process of training a model begins by splitting the clean data into two sets: training and testing. If there is sufficient data, it is also beneficial to create a validation set. The analysis revealed that a class imbalance in the target variable exists. Therefore, the `'stratify'` parameter was set to `'y'` (the target). It is worth noting that 10% of the entire dataset was reserved for the test set, while the remaining 90% was allocated for training.

3.2 Data Preprocessing

The network needs scaled, preprocessed training-testing sets before being fed. `X_train` should be fit and transformed, while `X_test` should only be transformed. A data transpose is also required for the `X` values for further matrix multiplication computations. Target values should be one-hot encoded.

3.3 Neural Network Architecture

The architecture of neural networks includes several key components. It starts with the input layer, which receives the raw data. This is followed by one or more hidden layers, where neurons process the input using activation functions like sigmoid, ReLU, or tanh to introduce non-linearity. Finally, the output layer generates the network's predictions or classifications based on the processed information. Weights connect the neurons and are adjusted during training to improve the network's performance.

3.4 Activation Functions

The two activation functions used in this project are mainly 'ReLU' (also its derivative for backpropagation) and 'Softmax'.

3.5 Methods

Each of the steps in a neural network, from initializing the parameters to updating them, was written as a single method, each responsible for a certain task. This is known as the Single Responsibility Principle (or SRP). Please note that the network is implemented from scratch (using NumPy and pure Python), since it is easier to retrieve and store the weights/biases for smaller datasets.

Methods of the Neural Network class include: `'initialize_parameters'`, `'forward_propagation'`, `'compute_cost'`, `'backward_propagation'`, `'initialize_adam'`, `'update_parameters'`

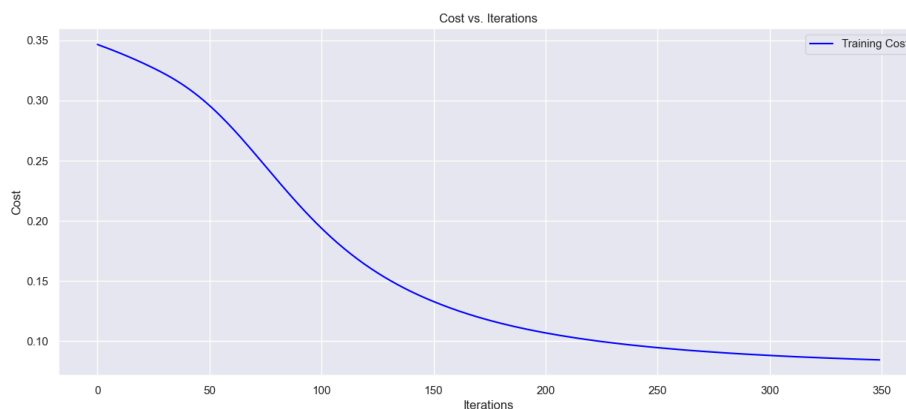
3.6 Class Neural Network

The network is written using a class and has methods that are in a separate file called `'methods.py'`. The general purpose of this class is to train the network and thus, get the best set of parameters (i.e., updated parameters retrieved from the Gradient Descent). The hyperparameters like `'learning_rate'`, `'decay_rate'`, `'penalty'`, and `'epochs'` were selected using `'GridSearchCV'`.

3.7 Result

The training resulted in 93.75% accuracy on the training set and 92.98% on the testing set. The model correctly classified 53 out of 57 test samples. Indices with the wrong predictions are array([2, 5, 40, 56]). Further analysis is required to determine whether the misclassified samples correspond to outliers or overlapping class regions.

A visual representation of the training's cost function has been provided below (Fig. 3):



3.8 Evaluation Metrics

Below, some useful evaluation metrics have been provided that show the overall network performance (Fig. 4). These include: *classification report & confusion matrix*

Classification Report:

Info	precision	recall	f1-score	support
malignant	0.90	0.90	0.90	21
benign	0.94	0.94	0.94	36
accuracy			0.93	57
macro avg	0.92	0.92	0.92	57
weighted avg	0.93	0.93	0.93	57

Table 4. Classification Report

Confusion Matrix:

```
[[19  2]
 [ 2 34]]
```

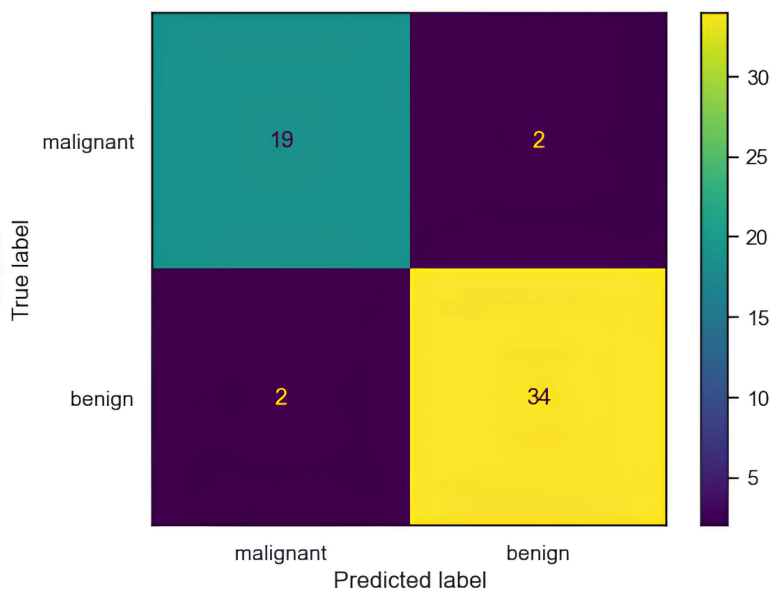


Figure 4. Confusion Matrix

The network achieved higher precision and recall for benign samples than for malignant samples.

The first 11 test data points along their classes are shown below:

Sample #	Prediction	Ground Truth
0	0	0
1	1	1
2	1	0
3	0	0
4	0	0
5	0	1
6	0	0
7	1	1
8	1	1
9	1	1
10	1	1

Table 5. Eleven Predictions along with their True Labels

3.9 Parameters

Since the main focus of this project is on designing a hardware-based simulated network in SystemVerilog, for the sake of simplicity, the weights/biases are converted to **fixed-point** rather than **floating-point**. Note that all parameters, weighted sum, and the activations are all required to return the maximum among them for input quantization. The first four tables show the parameters in floating-point format.

- a. Floating-point parameters:

First-layer weight matrix (W_1)

Hidden Neuron	worst smoothness	mean radius
h_1	0.05276146	0.04545019
h_2	0.87860578	1.4938967
h_3	0.04193307	0.03875046
h_4	0.84913852	1.46507294

First-layer bias vector (b_1)

Hidden Neuron	Bias
h_1	-0.0500322
h_2	0.93571265
h_3	-0.05000003
h_4	0.90332097

Second-layer weight matrix (W_2)

Output Neuron	h_1	h_2	h_3	h_4
O_1	-0.0343271	1.42355861	-0.05331349	1.46707236
O_2	0.01890401	-1.42651996	0.0436103	-1.45686225

Second-layer bias vector (b_2)

Output Neuron	Bias
O_1	-1.20539659
O_2	1.20539659

b. Fixed-point parameters:

First-layer weight matrix (W_1)

Hidden Neuron	worst smoothness	mean radius
h_1	216	186
h_2	3599	6119
h_3	172	159
h_4	3478	6001

First-layer bias vector (b_1)

Hidden Neuron	Bias
h_1	-205
h_2	3833
h_3	-205
h_4	3700

Second-layer weight matrix (W_2)

Output Neuron	h_1	h_2	h_3	h_4
O_1	-141	5831	-218	6009
O_2	77	-5843	179	-5967

Second-layer bias vector (b_2)

Output Neuron	Bias
O_1	-4937
O_2	4937

4. Fixed-Point Parameters

To make predictions and run inferences in HDL / SystemVerilog, the parameters needed to be converted from floating-point to fixed-point and stored in n-bit signed signals.

A **16-bit signed fixed-point** representation was selected because:

- Easy to implement in hardware
- Has a small memory footprint
- Provides sufficient precision for many machine-learning applications

To determine the fixed-point format, the following measurements should be computed in order:

- The maximum observed magnitude among parameters, X_{train} , X_{test} , weighted sums, and activations denoted as M
- The required number of integer bits, which is denoted as m
- The required number of fractional bits, which is denoted as n
- $Q_{m.n}$

In this work:

- $M = \max(|Z_2|) \approx 5.131$
- $2^m > M \Rightarrow m = \lceil \log_2(M) \rceil = \lceil \log_2(5.131) \rceil = 3$
- In 16 bits: $1 + m + n = 16$
- Therefore: $n = 12$

The computation yields **Q3.12**, which supports the range $[-8, 7.999755859375)$, while still preserving good precision using 12 fractional bits.

The corresponding scaling factor is:

$$Scale = 2^{12} = 4096$$

The floating-point values have been converted to fixed-point using the following formula:

- Quantization scale: $2^{12} = 4096$
- $X_{\text{fixed}} = \text{round}(x * 4096)$

It is worth mentioning that quantized inference using Q3.12 achieved the same testing accuracy (92.98%) as the floating-point implementation, confirming that Q3.12 provides sufficient numerical precision and dynamic range for the neural-network inference task.

Summary

In this phase, the implementation of a shallow neural network for the Wisconsin Breast Cancer Dataset was covered with a focus on converting trained parameters to fixed-point format for SystemVerilog. Data analysis and feature selection were conducted using correlation metrics, which resulted in the selection of "mean radius" and "worst smoothness" as key features with the lowest multicollinearity and highest correlation to the target.

By Kiana Jafari - Computer Engineering Student